

在汉字编码输入方案优化时，若使用了字根作为编码元素，则需要考虑选取哪些字根。除了极少数完全根据规则来决定字根选取与否的方案，大部分方案在选取字根上有一定自由度，因此也应该作为一种优化的自由度来调整。传统优化算法一般只考虑字根元素的布局，而如果结合字根的选取则能进一步提升性能。

[汉字自动拆分原理](#)讲解了给定字根集和拆分规则下的自动拆分。这个算法容易实现、扩展性强，但缺点是速度慢，不易提升性能。给定元素序列、libchai 完成一次编码和评测的时间一般在几十到几百微秒，但完整运行一次自动拆分算法需要几百毫秒，两者相差了至少 3 个数量级，因此用这样的方法来实现动态拆分是得不偿失的。

但是，与通用的自动拆分问题不同，在优化中遇到的自动拆分问题通常具有以下优点：

1. 通常并不是所有的字根都参与选取与否的决策，因为方案首先要保证拆分的直观性和系统性。以下我们把必须选取的字根称为「必要字根」，选取与否都可以的字根称为「可选字根」；
2. 优化中「拆分规则」本身不会参与变化，因此即使考虑到不同的字根选取的情况，通常一个部件也只有几种拆分方法。

下面提出一种充分利用以上两个优点来实现的快速自动拆分算法，它能够在选取的字根集发生变动的时候快速刷新拆分，并重新生成元素序列列表。这个过程的用时应该能够与静态拆分的编码生成处在同一量级。

字根集如何影响拆分结果？

我们首先回顾自动拆分的数学表述。对于部件 c 和字根集 R ，它的全部可能拆分 x 构成可行拆分集 W ，而其最优拆分 x^* 是使择优函数 $\mathcal{F}$ 最小化的那个拆分

$$x^* = \arg \min_{x \in W} \mathcal{F}(x)$$

显然这里 W 和 x^* 依赖于 c 和 R ，方便起见我们写作 $W[c, R]$ 和 $x^*[c, R]$ 。这里我们先固定 c ，考虑两个不同大小的字根集 R 和 R' ，并且他们满足 $R \subset R'$ （前者是后者的子集）。注意到以下特点：

1. $W[c, R] \subset W[c, R']$ 。这是因为当添加新的字根时，可能的拆分方式也会增加；
2. $\mathcal{F}(x^*[c, R]) \geq \mathcal{F}(x^*[c, R'])$ 。这是因为最优拆分总是使得择优函数最小化，在更大集合 $W[c, R']$ 上的最小值肯定比在它的子集 $W[c, R]$ 上的最小值更小。

在方案优化的过程中，我们通常会根据拆分的直观性、系统性来决定一个必要字根的集合 R_0 ，再根据一定的筛选标准找出一些可选字根，必要字根和可选字根的并集为 $R_{\max}$ 。因此在优化中字根集 R 变化的范围是 $R_0 \subset R \subset R_{\max}$ 。

在开始优化前，如果我们首先用 $R_{\max}$ 来运行自动拆分算法，对于每个部件 c 我们就得到了可行拆分集 $W[c, R_{\max}]$ 。根据之前的讨论，不管之后字根集 R 怎么变化，只要满足 $R \subset R_{\max}$ ，那么相应的最优拆分 $x^*[c, R]$ 就一定是 $W[c, R_{\max}]$ 中的一员。也就是说，字根集变化的时候，我们不需要生成任何新的拆分方式，只需要在之前生成过的拆分方式中选择对应于当前字根集的最优拆分即可。同时，不管字根集 R 怎么变化，对应的最优拆分 $x^*[c, R]$ 一定满足 $\mathcal{F}(x^*[c, R]) \leq \mathcal{F}(x^*[c, R_0])$ ，因此任何择优函数取值比 $\mathcal{F}(x^*[c, R_0])$ 还高的拆分在任何情况下都不会是最优拆分。也就是说，我们不需要保存完整的可行拆分集 $W[c, R_{\max}]$ ，我们只需要保存一小部分接下来可能会用得到的，把这个集合记作 $V[c, R_0, R_{\max}]$ ：

$$V[c, R_0, R_{\max}] = \{x | x \in W[c, R_{\max}] \text{ 并且 } \mathcal{F}(x) \leq \mathcal{F}(x^*[c, R_0])\}$$

不妨设这个集合是按 $\mathcal{F}$ 取值升序排序的。那么，不管字根集 R 怎么变化，我们只需要从这个集合的开头一项一项地检查拆分 x 是否全部由 R 中的字根构成：

- 如果是，那么这个拆分就是当前 R 对应的最优拆分 $x^*[c, R]$ ；
- 如果不是，那么这个拆分不属于当前 R 的可行拆分集，继续检查下一项。

实例

我们这里用「井」这个部件的拆分来演示上述分析。假设我们用 1, 2, 3, 4, 5 分别表示横竖撇点折，必要字根 $R_0 = \{ \text{丿, 升, 1, 2, 3, 4, 5} \}$ ，全部字根 $R_{\max} = R_0 \cup \{ \text{ㄣ, 开, 井} \}$ 。定义择优函数为

$$\mathcal{F}(x) = (\text{字根数量}(x), \text{符合笔顺}(x), \text{取大优先}(x))$$

那么

- 必要字根 R_0 下的可行拆分集为 $W[c, R_0] = \{(\cdot \cdot 1 \cdot \cdot), \dots, (4 \cdot 3 \cdot 1 \cdot 1 \cdot 3 \cdot 2)\}$, 最优拆分 $x^*[\cdot \cdot, R_0] = (\cdot \cdot 1 \cdot \cdot)$;
- 全部字根 $R_{\max}$ 下的可行拆分集为 $W[c, R_{\max}] = \{(\cdot \cdot), (\cdot \cdot \cdot \cdot \cdot), (\cdot \cdot \cdot \cdot), (\cdot \cdot 1 \cdot \cdot), \dots, (4 \cdot 3 \cdot 1 \cdot 1 \cdot 3 \cdot 2)\}$, 最优拆分 $x^*[\cdot \cdot, R_{\max}] = (\cdot \cdot)$;
- 上面的可行拆分集已经是根据 $\mathcal{F}$ 升序排序过的。根据前面的讨论, 我们不需要保存可行拆分集里的所有拆分, 只需要那些 $\mathcal{F}(x) \leq \mathcal{F}(x^*[c, R_0])$ 的拆分就可以。因此

$$V[\cdot \cdot, R_0, R_{\max}] = \{(\cdot \cdot), (\cdot \cdot \cdot \cdot \cdot), (\cdot \cdot \cdot \cdot), (\cdot \cdot 1 \cdot \cdot)\}$$

这样, 在字根集 R 发生变化时, 我们只需要按顺序检查这个有序集合, 就能得到对应于当前 R 的最优拆分, 例如:

- $R = R_0 \cup \{\cdot \cdot\}$, 此时第一项就全部由 R 中的字根构成, 最优拆分 $(\cdot \cdot)$;
- $R = R_0 \cup \{\cdot \cdot \cdot \cdot\}$, 此时第一项 $(\cdot \cdot)$ 不可行, 第二项 $(\cdot \cdot \cdot \cdot \cdot)$ 不可行, 第三项 $(\cdot \cdot \cdot \cdot)$ 可行, 因此最优拆分 $(\cdot \cdot \cdot \cdot)$;
- $R = R_0$, 此时第一项、第二项、第三项都不可行, 因此最优拆分 $(\cdot \cdot 1 \cdot \cdot)$ 。

算法表述

综上所述, 得到的算法如下:

1. 使用 $R_{\max}$ 运行自动拆分算法, 对于每个部件 c :
 1. 把所有可行拆分按照择优函数的大小从小到大排序, 其中第一项为 $x^*[c, R_{\max}]$, 将其称为拆分下界;
 2. 从前往后取若干个拆分, 一直取到 $x^*[c, R_0]$ 就不再继续往后取了, 将其称为拆分上界。拆分下界和拆分上界之间所有的可行拆分都存储起来得到一个有序集合 $V[c, R_0, R_{\max}]$;
2. 在优化时, 使用退火等搜索算法不断地把字根向字根集 R 中加入或从中删除。更新 R 之后, 对于每个部件 c :
 1. 从头开始遍历上述有序集合, 当遇到一个 x 满足 x 中的所有字根都属于 R 则停止遍历, 此时这个 x 就是对应当前 R 的最优拆分 $x^*[c, R]$ 。这是因为 $x^*[c, R]$ 一定存在于这个有序集合中, 且前面的拆分 (择优函数值较小的) 都不可行。
3. 根据新的拆分来更新所有包含这个部件 c 的词的分序列。

这个算法把自动拆分转化为了一个简单的遍历问题, 其复杂度仅仅取决于下界和上界之间拆分的数量, 一般不会很多。